

VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Programming Intergration Project

Artificial Intelligence

Project Proposal

TrendSpy: Universal Stock Price Interval Prediction Model

Instructor: Msc. Khuong Nguyen An

Student: Cuong Doan Phuong Hung ID 2310381

HO CHI MINH CITY, 6 DECEMBER 2025



Contents

1	Motivation & Problem Statement	1
1.1	The Problem	1
1.2	The Oppotunity	1
2	Primary Objective	2
2.1	Probabilistic Output	2
2.2	Alpha Capture	2
2.3	Universal Scalability	2
3	Technical Approach	2
3.1	Input	2
3.2	Output	3
3.3	Horizon	3
4	Implementation Roadmap	4
5	Expected Outcomes	5
5.1	Target Metrics	5
5.2	Deliverables	5



Executive Summary

TrendSpy is a quantitative finance project leveraging deep learning in PyTorch to forecast 5-day probabilistic price change intervals for over 50 major stocks using daily OHLCV data. The model outputs a **mean percentage change** (μ), an **uncertainty bound** (σ), and a **derived confidence score** (e.g., 62% chance of +0.2% to +1.0%), focusing on relative alpha through cross-sectional normalization. By combining LSTM for temporal processing, Transformer for contextual awareness, and a Gaussian output head, it addresses the limitations of point predictions in stochastic markets. The implementation follows a phased roadmap, starting with baseline viability and progressing to robust backtesting, targeting metrics like Information Coefficient (IC) > 0.03 and a long-short Sharpe Ratio > 0.5 . Deliverables include a daily-updatable pipeline and UI stock cards for tradeable insights.

1 Motivation & Problem Statement

1.1 The Problem

Traditional stock price prediction models, such as those relying on point estimates (e.g., a specific forecasted price), often fail in highly stochastic financial markets where uncertainty and noise dominate. These models frequently capture market “Beta” - general trends affecting all stocks, like broad economic shifts - rather than “Alpha,” which represents a stock’s relative outperformance against peers. This leads to unreliable predictions, especially over multi-day horizons, as they ignore probabilistic distributions and inter-stock dependencies. Additionally, without calibrated uncertainty, outputs lack trading utility, resulting in overconfident or vague signals that do not align with real-world risk management.

1.2 The Opportunity

Modern portfolio managers, traders, and quantitative analysts require “tradeable intelligence” in the form of probabilistic intervals that quantify expected returns, risks, and confidence levels. By shifting to interval predictions ($\mu \pm \sigma$), models can better capture market stochasticity, enabling decisions such as entering positions only when confidence exceeds 60% for positive returns. This project exploits deep learning’s ability to process multi-stock data universally, isolating alpha via techniques like cross-sectional Z-scores, which prevent the model from merely learning when the whole market rises or falls. The 5-day horizon reduces daily noise while providing actionable foresight, aligning with institutional strategies for relative value trading.

2 Primary Objective

2.1 Probabilistic Output

The core aim is to predict 5-day cumulative price change intervals, outputting μ (expected % return), σ (uncertainty bound as log variance), and a derived confidence score. Confidence is calculated as the probability the return falls within a threshold, such as $P(\text{return} > 0)$ using the cumulative distribution function (CDF) of a $\mathcal{N}(\mu, \sigma)$ distribution. This ensures outputs are calibrated and practical, allowing users to filter for high-probability trades.

2.2 Alpha Capture

To focus on relative performance, the model incorporates cross-sectional Z-scores, normalizing features and returns daily across the stock universe. This removes market beta, forcing the model to identify outliers that outperform or underperform peers. I use Z-scores to prevent the model from just learning when the whole market goes up, instead emphasizing true alpha through techniques like residual returns (Stock Return – Beta $\times$ Market Return) and a balanced loss function that prioritizes ranking alongside magnitude.

2.3 Universal Scalability

Design a single model trained simultaneously on many stocks, using a fixed universe for efficient batching and inter-stock learning. This enables daily updates via API ingestion and scalability to larger universes, ensuring timeliness and broad applicability without per-stock retraining.

3 Technical Approach

3.1 Input

Daily OHLCV data is transformed into stationary features for probabilistic 5-day predictions. Key preprocessing includes log returns ($\log(\text{Close}/\text{Prev_Close}) \times 100$) for additivity and symmetry, distance to simple moving average ($\text{Dist_SMA} = \text{Close}/\text{SMA} - 1$) for mean-reversion signals (with typical periods: 20, 60, 120), and additional indicators such as Relative Strength Index (RSI), Rate of Change (ROC), and rolling volatility for regime detection.

Each feature is cross-sectionally Z-scored (standardized) across the universe daily to isolate alpha and remove market-wide effects. Outliers are handled with winsorization. Synthetic index features, representing aggregated market context, are also included as additional signals. This



standardized pipeline transforms raw prices into robust, model-ready inputs for uncertainty-aware interval prediction.

3.2 Output

A hybrid model architecture is adopted to balance temporal forecasting and cross-sectional learning. The core consists of a temporal backbone (LSTM or GRU) to extract features from each stock's sequential data. These outputs are fed into Transformer layers, which apply attention across the set of stocks for each time step, enabling the model to recognize relative context, market regimes, and peer performance. Additionally, static stock-specific embeddings are included to allow differentiation between stocks beyond just their time series, communicating information such as sector or market capitalization.

The output head is fully probabilistic: it produces both a mean (μ) for the predicted return and a log-variance (σ) representing uncertainty, parameterizing a Gaussian distribution. A confidence score is then derived using the cumulative distribution function (CDF) of $\mathcal{N}(\mu, \sigma)$, e.g., $P(\text{return} > 0)$.

Training employs a blended loss function to balance accurate ranking (relative alpha) and precise probabilistic calibration. Specifically:

- **MarginRankingLoss** (weight α): Ensures the model ranks stocks correctly by predicted return, prioritizing selection of winners versus losers and supporting long-short strategies.
- **Gaussian Negative Log-Likelihood (GNLL)** (weight $1 - \alpha$): Penalizes poor fit in both mean and estimated uncertainty, ensuring μ is unbiased and σ is neither over- nor under-confident. This discourages trivial solutions that predict excessively wide intervals or collapse to mean-reversion.

This approach produces calibrated distributional estimates suitable for risk-aware trading signals, and robust to varying market conditions.

3.3 Horizon

The 5-day forecast horizon balances noise reduction (vs. 1-day) with actionable utility, enabling cumulative log return targets that capture meaningful trends while addressing label overlap through purged validation.

4 Implementation Roadmap

- **Phase 1: Baseline System (Proof of Concept)**

- Establish initial viability using a simple LSTM model trained on 8–15 representative stocks.
- Limit input features to raw log returns and basic indicators such as RSI, ROC, and rolling volatility.
- Train on 2010–2020 data using MSE loss to predict only the mean (μ) return.
- Evaluate performance on chronological splits, targeting MAE $< 2\%$ and Information Coefficient (IC) ≥ 0.01 .
- If metrics are unmet, halt advancement and prioritize data quality and repeatability.

- **Phase 2: Probabilistic Core (Gaussian Head & Calibration)**

- Extend the model to output both μ and uncertainty σ using a Gaussian output head.
- Optimize using Gaussian Negative Log-Likelihood (GNLL) loss.
- Add features including distance to SMA (Dist_SMA) and synthetic index signals.
- Calculate prediction confidence with `scipy.stats.norm.cdf`.
- Evaluate interval coverage (aiming for $\sim 68\%$ of true returns within $\pm\sigma$).
- Ensure Brier score < 0.25 .
- Address variance overconfidence with explicit regularization.

- **Phase 3: Cross-Sectional Alpha (Z-Scores & Ranking Loss)**

- Deploy a universal data loader for batching across all stocks and time slices.
- Introduce daily cross-sectional Z-scores and winsorization of inputs/targets.
- Shift the loss to a blend of MarginRanking and GNLL.
- Integrate static embeddings and basic cross-attention for peer context.
- Track key metrics: IC ≥ 0.02 – 0.03 and positive IC decay over days 1–5, confirming alpha extraction from relative normalization and model scalability.

- **Phase 4: Robust Backtesting (Purged Walk-Forward)**

- Upgrade validation to purged walk-forward (purging at least 5 days before test splits).
- Optionally incorporate residual return targets.



- Perform full long-short portfolio backtests (top/bottom selection) including slip-page/costs of 5–10bps.
- Benchmark Sharpe > 0.4 – 0.5 .
- Finalize daily inference and calibration CSVs for analysis.
- Ensure the framework is ready for production and real-time deployment.

5 Expected Outcomes

5.1 Target Metrics

Achieve the following quantifiable goals to ensure statistical robustness and economic viability:

- **MAE $< 1\%$** on predicted mean (μ), demonstrating high-precision forecasts.
- **Interval Coverage:** 68% of realized returns should fall within $[\mu \pm \sigma]$, confirming well-calibrated uncertainty estimation.
- **Calibration Brier Score < 0.2** , indicating reliable probabilistic outputs.
- **Information Coefficient (IC) > 0.03** on cross-sectional ranks, reflecting meaningful signal extraction.
- **Positive IC Decay** over five forecast days, providing actionable horizon insight.
- **Long-Short Sharpe Ratio > 0.5** in walk-forward backtests, validating economic utility after transaction costs.

5.2 Deliverables

- A **deployable PyTorch pipeline** for automated, daily model retraining and inference.
- **UI elements** including interactive stock cards with percent change bars, confidence scores, and upper/lower predictive bounds for end-user interpretability.
- **Comprehensive logging:** Daily CSVs of [Date, Ticker, μ , σ , Actual] to support both monitoring and advanced analysis.
- **Optional extensions:**
 - Full Transformer-based model integration for sequence modeling.
 - Hyperparameter optimization using tools like Optuna.
 - Sentiment signal ingestion (e.g., via `x_semantic_search`) for additional alpha.